

GUÍA TÉCNICA

Entornos Virtuales con venv.

¿Qué es venv?

Venv es un módulo incorporado en Python (a partir de la versión 3.3) que permite crear entornos virtuales aislados. Un entorno virtual es un directorio independiente que contiene una instalación específica de Python y sus paquetes, separada del sistema global. Esto permite gestionar dependencias de proyectos de manera individual, evitando conflictos entre versiones de librerías.

Cuando se activa un entorno virtual, cualquier paquete que se instale se colocará en el directorio privado de ese entorno, no en la instalación global de Python del sistema. Este aislamiento ofrece varios beneficios:

- Puedes utilizar diferentes versiones del mismo paquete para diferentes proyectos.
- Puedes compartir fácilmente la lista exacta de paquetes requeridos con otros.
- Puedes mantener tu instalación global de Python limpia y organizada.

1. Creación de un entorno virtual

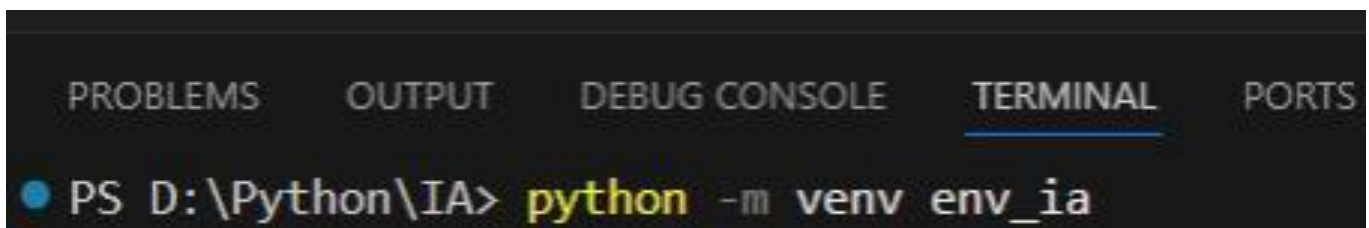
Para crear un entorno virtual, abre tu terminal y navega al directorio de tu proyecto. Luego, ejecuta:

```
Python -m venv nombre_del_entorno
```

Reemplaza nombre_del_entorno por el nombre que desees (por ejemplo, env, venv, .venv).

Ejemplo:

```
Python3 -m venv .venv
```



2. Activación del entorno virtual

Una vez creado, debes activar el entorno virtual para usarlo.

- En Linux/macOS:

Source nombre_del_entorno/bin/activate

- En Windows (PowerShell):

Nombre_del_entorno\Scripts\Activate.ps1

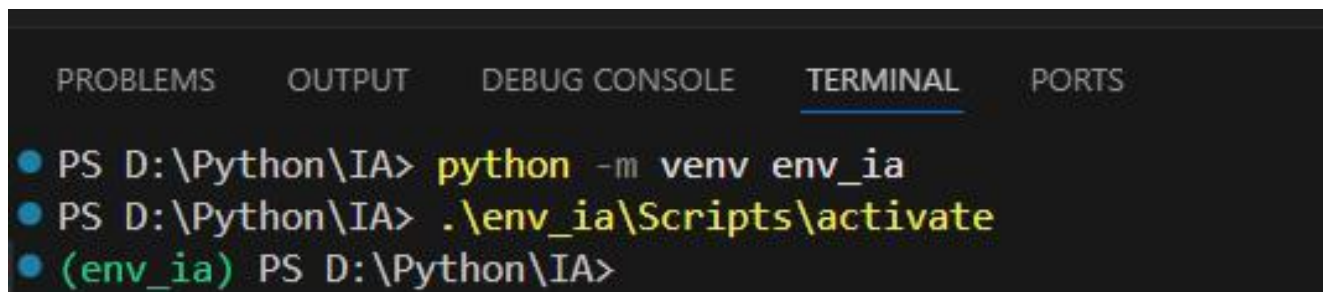
- En Windows (CMD):

Nombre_del_entorno\Scripts\activate.bat



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Python\IA> python -m venv env_ia
PS D:\Python\IA> .\env_ia\Scripts\activate
```

Si se ha activado el entorno virtual correctamente, nos mostrará en el prompt en color verde el nombre del entorno virtual, entre paréntesis:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\Python\IA> python -m venv env_ia
PS D:\Python\IA> .\env_ia\Scripts\activate
(env_ia) PS D:\Python\IA>
```

3. Desactivación del entorno virtual

Para salir del entorno virtual y volver al entorno global de Python, ejecuta:

Deactivate

4. Instalación de paquetes en el entorno virtual

Con el entorno activado, puedes instalar paquetes con pip. Estos paquetes se instalarán solo en el entorno virtual, sin afectar al sistema global.

Pip install nombre_del_paquete

Desde aquí podremos instalar todas las dependencias que necesitemos.

Ejemplo:

Pip install requests

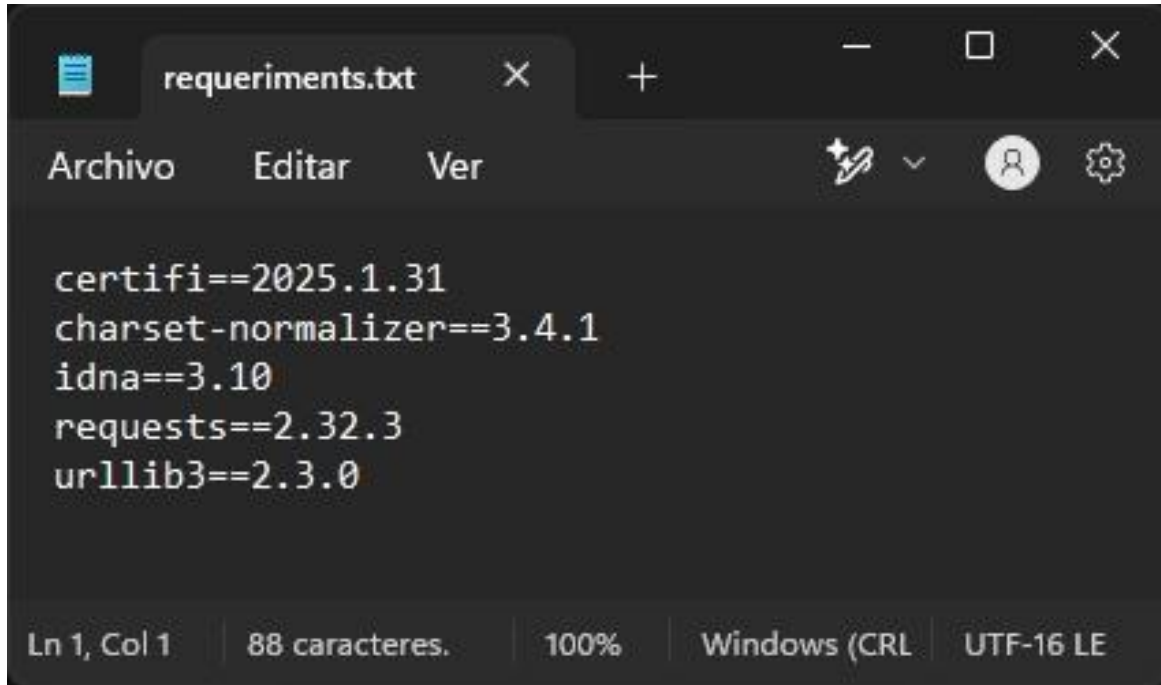
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS D:\Python\IA> python -m venv env_ia
● PS D:\Python\IA> .\env_ia\Scripts\activate
● (env_ia) PS D:\Python\IA> pip install requests
Collecting requests
  Using cached requests-2.32.3-py3-none-any.whl.metadata (4.6 kB)
Collecting charset-normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.4.1-cp313-cp313-win_amd64.whl.metadata (36 kB)
Collecting idna<4,>=2.5 (from requests)
  Using cached idna-3.10-py3-none-any.whl.metadata (10 kB)
```

Una vez que tengamos instalados todos los paquetes que podamos necesitar para nuestro proyecto Python, es recomendable crear el fichero requeriments.txt, que contendrá todas las dependencias de nuestro entorno y nos servirá para desplegarlo posteriormente con un solo comando. Para crear el fichero requeriments.txt, desde la terminal, con nuestro entorno activado, ejecutaremos:

Pip freeze > requeriments.txt

```
● (env_ia) PS D:\Python\IA> pip freeze > requeriments.txt
○ (env_ia) PS D:\Python\IA>
```

En realidad, lo que hace el comando anterior es buscar todos los paquetes instalados en nuestro entorno virtual y agregarlos al fichero `requeriments.txt`, que quedará con este contenido (variará en función de los paquetes instalados):

A screenshot of a code editor window titled 'requeriments.txt'. The editor has a dark theme and a menu bar with 'Archivo', 'Editar', and 'Ver'. The main text area contains five lines of package specifications: 'certifi==2025.1.31', 'charset-normalizer==3.4.1', 'idna==3.10', 'requests==2.32.3', and 'urllib3==2.3.0'. The status bar at the bottom shows 'Ln 1, Col 1', '88 caracteres.', '100%', 'Windows (CRLF)', and 'UTF-16 LE'.

```
certifi==2025.1.31
charset-normalizer==3.4.1
idna==3.10
requests==2.32.3
urllib3==2.3.0
```

Si, posteriormente, necesitamos desplegar el entorno virtual con sus paquetes en otro equipo, solo tendremos que pasar los ficheros `.py` y el fichero `requeriments.txt`, y desde el entorno virtual activado ejecutar:

```
Pip install -r requirement.txt
```

Que leerá el contenido del fichero `requeriments.txt` e instalará todos sus paquetes.

Crearemos todos los ficheros `.py` de nuestro proyecto en la carpeta raíz, donde quedó el fichero `requeriments.txt`:

6. Consejos clave.

1. **Nombres consistentes:** Usa nombres como `.venv` o `venv` para que sea fácil reconocer el entorno.
2. **Ignorar en Git:** Añade el nombre del entorno virtual al archivo `.gitignore` para evitar subirlo al repositorio.
3. **Guardar dependencias:** Siempre crea y mantén actualizado un archivo `requirements.txt` con:

```
Pip freeze > requirements.txt
```
4. **Recrear entornos:** Para reproducir un entorno en otra máquina, usa:

```
Pip install -r requirements.txt
```
5. **Entornos por proyecto:** Cada proyecto debe tener su propio entorno virtual.

7. Conclusión

Los entornos virtuales `venv` son esenciales para desarrollar proyectos en Python de manera organizada, reproducible y libre de conflictos. Siguiendo esta guía podrás aislar dependencias, colaborar eficientemente y mantener tu sistema global limpio.